Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Qual è l'ordine dei bucket di origine e importanza, dal **meno** al **più** prioritario?

- [ ] A — Autore, utente, user agent, autore `!important`, utente `!important`, user agent `!important`.
- [x] B — User agent, utente, autore, autore `!important`, utente `!important`, user agent `!important`.
- [ ] C — User agent, autore, utente, utente `!important`, autore `!important`, user agent `!important`.
- [ ] D — User agent `!important`, utente `!important`, autore `!important`, autore, utente, user agent.

Risposta: B. Fra le dichiarazioni normali vince l'autore (che sovrascrive gli stili predefiniti del browser); fra le `!important` l'ordine si **inverte**: le esigenze dell'utente (per esempio l'alto contrasto per accessibilità) devono poter battere quelle del sito. *(Dispensa: cap. 3, «Origine e importanza»)*

Es. **2.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const nome = "Ada";
2 const anni = 36;
3 console.log(`${nome} ha ${anni + 1} anni`);
4 console.log('${nome} ha ${anni + 1} anni');
```

- [ ] A — `Ada ha 37 anni`, `Ada ha 361 anni`.
- [x] B — `Ada ha 37 anni`, `${nome} ha ${anni + 1} anni`.
- [ ] C — Viene sollevato un `SyntaxError`.
- [ ] D — `Ada ha 361 anni`, `${nome} ha ${anni + 1} anni`.

Risposta: B. Solo i *template literal*, delimitati dagli accenti gravi, interpretano le espressioni `${...}`; fra apici singoli o doppi il testo resta letterale. Dentro `${}` si valuta un'espressione: `anni + 1` è 37. *(Dispensa: cap. 5, «Funzioni»)*

Es. **3.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const voti = new Map([["web", 30], ["reti", 27]]);
2 voti.set("web", 28);
3 for (const [esame, voto] of voti) {
4   console.log(esame + ": " + voto);
5 }
```

- [ ] A — `web: 30`, `reti: 27`.
- [ ] B — `reti: 27`, `web: 28`.
- [ ] C — `0: web,28`, `1: reti,27`.
- [x] D — `web: 28`, `reti: 27`.

Risposta: D. Iterando una `Map` con `for...of` si ottengono coppie `[chiave, valore]`, che si possono destrutturare. Le chiavi restano nell'ordine di inserimento; `set` su una chiave esistente ne aggiorna il valore senza spostarla. *(Dispensa: cap. 6, «Iterare su una mappa»)*

SOLUZIONI

Es. **4.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<ul><li>B</li></ul>
<script>
  function li(testo) {
    const e = document.createElement("li");
    e.textContent = testo;
    return e;
  }
  const ul = document.querySelector("ul");
  ul.before("X");
  ul.prepend("Z");
  ul.after("Y");
  console.log(ul.children.length, ul.childNodes.length);
  console.log(ul.textContent);
</script>
```

- A 1 2, BZ.
- **B** 1 2, ZB.
- C 1 1, ZB.
- D 2 3, ZB.
- E 1 4, ZXBY.
- F 2 2, ZB.

Risposta: B. `prepend` e `append` inseriscono **dentro** l'elemento, all'inizio e in fondo; `before` e `after` **fuori**, subito prima e subito dopo di esso. Le stringhe diventano nodi di testo: contano in `childNodes` ma non in `children`, che contiene solo elementi. `textContent` concatena il testo di tutti i discendenti, nell'ordine. *(Dispensa: cap. 7, «Dove: i quattro punti di inserimento»)*

Es. **5.** **(2 punti)** Nella sezione *Tempistiche* della scheda *Rete* dei DevTools, una richiesta ha una fase *In attesa* molto lunga. Che cosa indica?

- A Che la risposta era molto grande da scaricare.
- B Che la risoluzione DNS è stata lenta.
- C Che la richiesta è rimasta in coda nel browser perché non c'era connessione.
- D Che il browser ha impiegato molto tempo a disegnare la pagina.
- E Che la negoziazione TLS è fallita.
- **F** Che il server ha impiegato molto tempo a elaborare la richiesta prima di inviare il primo byte della risposta.

Risposta: F. *In attesa* è il tempo fra l'invio della richiesta e l'arrivo del primo byte della risposta (*Time To First Byte*): il tempo di elaborazione del server. DNS, connessione e TLS hanno barre proprie, e il tempo per scaricare il corpo è la *Ricezione*. *(Dispensa: cap. 9, «Dal clic alla pagina: il viaggio completo»)*

Es. **6.** **(2 punti)** Un server serve soltanto i file della sua *document root*. Quali delle seguenti esigenze **non** si possono soddisfare in questo modo?

- A Servire un foglio di stile CSS condiviso da tutte le pagine.
- B Servire la stessa pagina a tutti gli utenti, in HTTPS.
- C Aggiornare il contenuto della pagina nel browser con JavaScript.
- D Servire immagini e script JavaScript.
- **E** Registrare un ordine in seguito all'invio di un form.
- **F** Personalizzare una pagina per l'utente che la richiede.

Risposte: E, F. Un server statico restituisce sempre lo stesso documento: niente personalizzazione, nessuna azione in seguito a una richiesta, e una struttura comune da ripetere in ogni file (per Wikipedia, terabyte di dati duplicati e milioni di file da rigenerare per cambiare la barra di navigazione). Il dinamismo di JavaScript avviene dentro il browser. *(Dispensa: cap. 10, «I limiti delle web app statiche»)*

SOLUZIONI

Es. **7.** **(2 punti)** A che cosa serve `nodemon`?

- [ ] A monitorare il traffico di rete del server.
- [ ] B mostrare le variabili in scope durante il debug.
- [ ] C scaricare le dipendenze mancanti all'avvio.
- [ ] D installare e cambiare le versioni di Node.js.
- [ ] E eseguire il server in più processi paralleli.
- [x] **F** riavviare automaticamente il server quando cambia il codice sorgente.

Risposta: F. Durante lo sviluppo, `nodemon app.js` al posto di `node app.js` sorveglia i file (`js`, `mjs`, `cjs`, `json`) e riavvia il processo a ogni modifica (*live reloading*). Per le versioni di Node si usano gestori come `nvm`. *(Dispensa: cap. 11, «Ricaricamento automatico»)*

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono, nell'ordine, le richieste `GET /`, `GET /nonesiste`, `GET /todo` e `GET /todo` con l'header `Cookie: login=si`?

```
1  import express from "express";
2
3  const app = express();
4  const todoRouter = express.Router();
5
6  function richiediLogin(req, res, next) {
7    if (req.headers.cookie === "login=si") next();
8    else res.redirect("/auth");
9  }
10 todoRouter.get("/todo", richiediLogin, (req, res) => res.send("La lista"));
11 app.get("/", (req, res) => res.send("Home"));
12 app.use(todoRouter);
13 app.get("/*splat", (req, res) => res.status(404).send("Non trovato"));
14
15 app.listen(3000);
```

- [x] **A** `200 Home, 404 Non trovato, 302 -> /auth, 200 La lista.`
- [ ] B `200 Home, 404 Non trovato, 302 -> /auth, 302 -> /auth.`
- [ ] C `200 Home, 404 Non trovato, 404 Non trovato, 200 La lista.`
- [ ] D `302 -> /auth, 302 -> /auth, 302 -> /auth, 200 La lista.`
- [ ] E `200 Home, 404 Non trovato, 200 La lista, 200 La lista.`
- [ ] F `200 Home, 302 -> /auth, 302 -> /auth, 200 La lista.`

Risposta: A. Un router montato senza percorso (`app.use(todoRouter)`) riceve **tutte** le richieste che le rotte precedenti non hanno gestito, e `todoRouter.use(mw)` senza percorso applica il middleware a tutte quelle che entrano nel router: anche `/nonesiste` verrebbe intercettata e reindirizzata al login invece di arrivare al *catch-all*. Il problema sparisce dando un percorso al middleware, montando il router sotto un prefisso o applicando il middleware rotta per rotta. *(Dispensa: cap. 16, «Una trappola: il middleware di un router»)*

Es. **9.** **(5 punti)** Si descrivano le differenze fra l'invio di un form con il metodo `GET` e con il metodo `POST`, indicando in quali casi conviene usare l'uno o l'altro.

Traccia della risposta.

- `GET`: i dati diventano la query della URL (`?nome=valore&...`), visibile, salvabile nei segnalibri, registrata in cronologia e nei log.
- `POST`: i dati viaggiano nel corpo della richiesta, codificati allo stesso modo (`application/x-www-form-urlencoded`).
- `GET` per operazioni che leggono (ricerche, filtri): è sicura e ripetibile.
- `POST` per operazioni che modificano lo stato (registrazioni, acquisti) o per dati lunghi; nessuno dei due metodi cifra i dati: per questo serve HTTPS.

(Dispensa: cap. 2, «Invio con GET»)

SOLUZIONI

Es. **10.** **(5 punti)** Si confrontino **Flexbox** e **Grid**: a che cosa servono, quali sono i concetti principali di ciascuno e quando conviene usare l'uno o l'altro.

Traccia della risposta.

- Flexbox dispone gli elementi lungo **una** dimensione (riga o colonna): asse principale e trasversale, `justify-content`, `align-items`, `flex-wrap`.
- Grid dispone gli elementi in **due** dimensioni: `grid-template-columns`/`rows`, unità `fr`, aree con nome (`grid-template-areas`, `grid-area`).
- In entrambi i casi gli stili si applicano al contenitore e i figli diretti diventano item.
- Flexbox per barre, menu, allineamenti di componenti; Grid per l'impaginazione complessiva della pagina. Si possono combinare.

(Dispensa: cap. 4, «Flexbox» e «Grid»)

Es. **11.** **(5 punti)** Un'applicazione memorizza lo stato nei cookie `auth` e `username`. Si descriva quali header si scambiano browser e server al login e nelle richieste successive, e si spieghi perché questo approccio non è sicuro.

Traccia della risposta.

- Il form invia le credenziali con una `POST`; se sono giuste il server risponde con un reindirizzamento (`302 Found`, `Location: /`) e due header `Set-Cookie` (`auth=true; max-age=3600` e `username=luigi; max-age=3600`), altrimenti ripropone il login con `401`.
- Il browser memorizza i cookie e da quel momento li allega a ogni richiesta allo stesso sito in un unico header `Cookie: auth=true; username=luigi`, che il server deve analizzare (coppie separate da `;`, nome e valore da `=`).
- Le pagine protette controllano `auth`: se manca rispondono `401` o reindirizzano al login.
- I cookie sono controllati dal client: con i DevTools si cambia `username` o ci si imposta `auth=true` senza login. I cookie firmati rivelano la manomissione, ma i dati restano sul client: la soluzione sono le sessioni web.

(Dispensa: cap. 13, «Un approccio ingenuo: i cookie» e «I problemi dell'approccio a cookie»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	B	3	origine e importanza
2	B	5	funzioni
3	D	6	Map e Set
4	B	7	modificare il DOM
5	F	9	HTTPS
6	E, F	10	pagine statiche e dinamiche
7	F	11	strumenti di sviluppo
8	A	16	autenticazione
9	aperta	2	form: invio e codifica
10	aperta	4	Flexbox
11	aperta	13	lo stato nei cookie

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI